

Guia de Comandos Linux

1. Comandos de Navegação de Diretórios

Estes comandos permitem que você se mova e entenda onde está no seu sistema de arquivos.

pwd (print working directory)

Função: Exibe o caminho completo do diretório atual onde você se encontra. É como perguntar "Onde estou agora?".

```
$ pwd  
/home/<seu_usuario>
```

Sempre que aparecer <seu_usuario> nos exemplos deste guia, substitua pelo nome do seu próprio usuário no sistema (não existe um usuário fixo chamado assim). Ao rodar pwd no seu terminal, ele mostrará algo como /home/joao ou /home/maria, dependendo de como sua conta foi criada. Sempre que aparecer ~, ele representa exatamente esse mesmo diretório home do seu usuário.

ls (list)

Função: Lista o conteúdo de um diretório. Sem argumentos, lista o conteúdo do diretório atual.

Principais tags:

- **-l** ou **--long**: lista em formato longo, mostrando permissões, proprietário, grupo, tamanho, data de modificação e nome do arquivo.
- **-a** ou **--all**: inclui arquivos e diretórios ocultos (os que começam com ponto, ex: **.bashrc**).
- **-h** ou **--human-readable**: mostra tamanhos em formato legível (KB, MB, GB). Usado junto com **-l**.
- **-R** ou **--recursive**: lista o conteúdo dos subdiretórios recursivamente.

```
$ ls      # Lista o conteúdo do diretório atual  
$ ls -l   # Lista com detalhes (formato longo)  
$ ls -la  # Lista tudo (incluindo ocultos) em formato longo  
$ ls -lh  # Lista com detalhes e tamanhos legíveis  
$ ls -R /etc/ # Lista recursivamente /etc/ (pode ser grande!)
```

cd (change directory)

Função: Muda o diretório de trabalho atual. É o comando mais usado para navegar.

Principais tags:

- **..**: vai para o diretório pai (um nível acima).
- **~**: vai para o diretório home do usuário.
- **-**: retorna ao diretório anterior.

```
$ mkdir -p Documentos          # garante que o diretório existe antes de entrar nele
$ cd /home/<seu_usuario>/Documentos  # Navega para o diretório (troque <seu_usuario>
pelo seu)
$ cd ..                        # Volta um diretório
$ cd ~                        # Vai para o seu diretório home
$ cd                          # Mesmo que 'cd ~'
$ cd -                        # Retorna ao último diretório visitado
```

2. Comandos de Manipulação de Arquivos e Diretórios

Estes comandos permitem que você crie, mova, copie e apague arquivos e diretórios. A partir daqui, os exemplos continuam no mesmo diretório de trabalho (`/home/<seu_usuario>`), então cada arquivo criado fica disponível para os comandos seguintes.

mkdir (make directory)

Função: Cria um novo diretório (pasta).

Principais tags:

- `-p` ou `--parents`: cria diretórios pais conforme necessário. Útil para criar estruturas aninhadas de uma vez.

```
$ mkdir meus_documentos      # Cria 'meus_documentos'
$ mkdir -p projetos/2026/janeiro # Cria 'projetos', '2026' e
                                # 'janeiro' um dentro do outro
```

rmdir (remove directory)

Função: Remove um diretório, mas apenas se ele estiver vazio.

```
$ mkdir diretoriovazio
$ rmdir diretoriovazio  # Remove o diretório vazio (criado na linha acima)

$ mkdir diretoriocomarquivos
$ touch diretoriocomarquivos/arquivo.txt
$ rmdir diretoriocomarquivos
# Isso falha, pois o diretório não está vazio
```

touch

Função: Cria um novo arquivo vazio. Se o arquivo já existir, atualiza a data e hora da última modificação.

```
$ touch novo_arquivo.txt    # Cria um arquivo vazio
$ touch relatorio_diario.log # Cria (ou atualiza o timestamp, se já existir)
```

cp (copy)

Função: Copia arquivos e diretórios.

Principais tags:

- `-r` ou `--recursive`: obrigatório para copiar diretórios e seu conteúdo.
- `-v` ou `--verbose`: mostra o progresso da cópia.
- `-i` ou `--interactive`: pergunta antes de sobrescrever um arquivo existente.

```
$ touch arquivo_original.txt
$ cp arquivo_original.txt copia_arquivo.txt
# Copia 'arquivo_original.txt' (recém-criado) para 'copia_arquivo.txt'

$ touch documento.pdf
$ mkdir -p ~/Backup
$ cp documento.pdf ~/Backup/
# Copia 'documento.pdf' para o diretório 'Backup' na home (já criado acima)

$ mkdir -p meu_projeto
$ touch meu_projeto/app.py
$ mkdir -p ~/ProjetosBackup
$ cp -r meu_projeto/ ~/ProjetosBackup/
# Copia o diretório 'meu_projeto' e todo o seu conteúdo
```

mv (move)

Função: Move arquivos e diretórios de um local para outro. Também é usado para renomear.

Principais tags:

- **-v** ou **--verbose**: mostra o que está sendo movido/renomeado.
- **-i** ou **--interactive**: pergunta antes de sobrescrever.

```
$ touch arquivo_antigo.txt
$ mv arquivo_antigo.txt arquivo_novo.txt
# Renomeia 'arquivo_antigo.txt' (recém-criado) para 'arquivo_novo.txt'

$ touch imagem.jpg
$ mkdir -p ~/Imagens
$ mv imagem.jpg ~/Imagens/
# Move 'imagem.jpg' para o diretório 'Imagens'

$ mkdir -p projeto_antigo
$ mv projeto_antigo/ projeto_revisado/
# Renomeia o diretório 'projeto_antigo' (recém-criado) para 'projeto_revisado'
```

rm (remove)

Função: Remove (apaga) arquivos e diretórios. Use com extrema cautela — arquivos removidos geralmente não vão para a lixeira e são difíceis de recuperar!

Principais tags:

- **-r** ou **--recursive**: obrigatório para remover diretórios e seu conteúdo (mesmo que não estejam vazios).

- **-f** ou **--force**: força a remoção, sem pedir confirmação. CUIDADO REDOBRADO ao usar **rm -rf**!
- **-i** ou **--interactive**: pergunta antes de remover cada arquivo. (Boa prática usar com **-r**)

```
$ touch arquivo_indesejado.txt
$ rm arquivo_indesejado.txt
# Remove o arquivo recém-criado

$ touch arquivo_para_remover.txt
$ rm -i arquivo_para_remover.txt
# Pergunta antes de remover o arquivo

$ mkdir -p diretorio_cheio
$ touch diretorio_cheio/a.txt diretorio_cheio/b.txt
$ rm -r diretorio_cheio/
# Remove o diretório e todo o seu conteúdo

$ mkdir -p arquivos_temporarios
$ touch arquivos_temporarios/tmp1 arquivos_temporarios/tmp2
$ rm -rf arquivos_temporarios/
# Remove sem pedir confirmação. Use com EXTREMO CUIDADO!
```

3. Comandos de Visualização e Edição de Conteúdo

Estes comandos permitem ver o que está dentro dos arquivos e fazer edições básicas diretamente no terminal.

cat (concatenate)

Função: Exibe o conteúdo de um arquivo diretamente no terminal. Mais adequado para arquivos pequenos, pois mostra tudo de uma vez.

```
$ echo "Este é um texto de exemplo." > meu_texto.txt
$ cat meu_texto.txt
# Exibe o conteúdo de 'meu_texto.txt'

$ echo "Conteúdo do arquivo 1" > arquivo1.txt
$ echo "Conteúdo do arquivo 2" > arquivo2.txt
$ cat arquivo1.txt arquivo2.txt
# Concatena e exibe o conteúdo de ambos os arquivos
```

less

Função: Permite visualizar o conteúdo de arquivos grandes página por página. Você pode rolar, pesquisar e sair sem carregar o arquivo inteiro na memória.

Principais tags:

- **q**: sai.
- **Espaço** ou **Page Down**: avança uma tela.
- **b** ou **Page Up**: retrocede uma tela.
- **/texto**: pesquisa por texto dentro do arquivo.

```
$ less /var/log/syslog # Visualiza o arquivo de log do sistema (já existe no S.O.)
```

head

Função: Exibe as primeiras linhas de um arquivo (por padrão, as 10 primeiras).

Principais tags:

- **-n <número>**: especifica o número de linhas a serem exibidas.

```
$ seq 1 20 > meu_config.conf # Gera um arquivo com 20 linhas de exemplo
$ head meu_config.conf      # Exibe as 10 primeiras linhas
$ head -n 5 meu_config.conf  # Exibe as 5 primeiras linhas
```

tail

Função: Exibe as últimas linhas de um arquivo (por padrão, as 10 últimas). Muito útil para arquivos de log em tempo real.

Principais tags:

- **-n <número>**: especifica o número de linhas a serem exibidas.
- **-f** ou **--follow**: "segue" o arquivo, exibindo novas linhas conforme são adicionadas (ideal para logs). Para sair, pressione **Ctrl+C**.

```
$ seq 1 20 > meu_log.log    # Gera um arquivo de exemplo com 20 linhas
$ tail meu_log.log          # Exibe as 10 últimas linhas
$ tail -n 20 meu_log.log    # Exibe as 20 últimas linhas
$ tail -f /var/log/auth.log # Monitora o log em tempo real (arquivo já existe no S.O.)
```

nano (e vi/vim)

Função: Editores de texto de linha de comando. O **nano** é mais amigável para iniciantes, com atalhos de teclado exibidos na parte inferior. O **vi** (ou **vim**) é mais poderoso, mas com curva de aprendizado mais íngreme.

Principais tags:

- **Ctrl+X**: sair.
- **Ctrl+O**: salvar.
- **Ctrl+W**: buscar.

```
$ nano novo_script.sh # Abre (e cria, se não existir) 'novo_script.sh' no nano
```

Dentro do **nano**, digite seu texto e use **Ctrl+O** para salvar e **Ctrl+X** para sair.

4. Outros Comandos Úteis

find

Função: Comando poderoso para encontrar arquivos e diretórios com base em vários critérios (nome, tipo, tamanho, data etc.). Pesquisa o sistema de arquivos em tempo real.

Principais tags:

- **-name "padrão"**: encontra arquivos com o nome correspondente.
- **-type f**: procura apenas arquivos.
- **-type d**: procura apenas diretórios.
- **-size +1G**: procura arquivos maiores que 1 Gigabyte.
- **-exec comando {} \;**: executa um comando nos arquivos encontrados.

```
$ touch relatorio_diario.log # já criado na seção 2, incluído aqui para o exemplo funcionar isoladamente
$ find . -name "*.log"
# Encontra todos os arquivos .log no diretório atual e subdiretórios

$ touch config_2026.bak
$ find /home/<seu_usuario> -type f -name "config*.bak"
# Procura arquivos de backup de configuração na home (agora existe pelo menos um)

$ find /tmp -type f -mtime +7 -delete
# Apaga arquivos em /tmp não modificados há 7+ dias (CUIDADO!)
```

grep

Função: Filtra e exibe linhas de texto que correspondem a um determinado padrão. Muito usado com o "pipe" (|) para filtrar a saída de outros comandos.

Principais tags:

- **-i** ou **--ignore-case**: ignora maiúsculas/minúsculas.
- **-r** ou **--recursive**: pesquisa recursivamente em diretórios.
- **-n** ou **--line-number**: mostra o número da linha onde o padrão foi encontrado.

```
$ grep "erro" /var/log/syslog
# Procura a palavra 'erro' no arquivo de log do sistema (já existe no S.O.)

$ ls -l | grep ".txt"
# Lista apenas os arquivos .txt na saída de 'ls -l' (usa os .txt criados nas seções anteriores)

$ echo "def funcao_principal():" > meu_projeto/app.py
$ grep -rn "funcao_principal" meu_projeto/
# Procura recursivamente pela função (agora presente em meu_projeto/app.py) e mostra o nº da linha
```

man (manual)

Função: Exibe o manual (documentação) de um comando. Indispensável para aprender todas as opções e usos de qualquer comando.

```
$ man ls # Abre o manual do comando 'ls'
$ man cp # Abre o manual do comando 'cp'
```

Dentro do **man**, use as setas para rolar, **Espaço** para a próxima página, **b** para a página anterior, e **q** para sair.

5. Exercícios Práticos I

Para realmente dominar esses comandos, a prática é fundamental! Crie um diretório de teste e divirta-se — tudo dentro da sua VM Ubuntu no VirtualBox.

1. Preparação

- Crie um diretório para seus exercícios: `mkdir ~/exercicios_linux`
- Navegue até o diretório: `cd ~/exercicios_linux`

2. Navegação

- Confirme que você está no diretório correto usando `pwd`.
- Crie um subdiretório chamado `documentos`.
- Entre no diretório `documentos`.
- Volte para `exercicios_linux` usando `cd ..`
- Vá para sua home usando `cd ~`
- Volte para `exercicios_linux` usando `cd -`

3. Criação e Manipulação de Arquivos

- Crie três arquivos vazios em `exercicios_linux`: `lista.txt`, `notas.md`, `backup_config.conf`
- Copie `lista.txt` para `documentos/lista_copia.txt`
- Mova `notas.md` para o diretório `documentos`
- Renomeie `backup_config.conf` para `config.old`
- Tente criar um diretório `meu_projeto/src/main` com um único comando. **Dica:** se o comando falhar com "No such file or directory", pesquise qual parâmetro do `mkdir` permite criar diretórios pai automaticamente.

4. Visualização e Edição

- Adicione linhas ao `lista.txt` usando `echo "Primeira linha" > lista.txt` e `echo "Segunda linha" >> lista.txt`
- Exiba o conteúdo de `lista.txt` usando `cat`.
- Exiba apenas a primeira linha de `lista.txt` usando `head -n 1 lista.txt`.
- Use `nano` para abrir e adicionar mais uma linha ao `notas.md` (dentro de `documentos`). Salve e saia.
- Use `tail` para verificar a última linha adicionada em `notas.md`

5. Busca e Filtragem

- Dentro de `exercicios_linux`, crie um arquivo `log_sistema.txt` e adicione as linhas:
 - `echo "INFO: Sistema iniciado." > log_sistema.txt`
 - `echo "ERRO: Falha de conexão." >> log_sistema.txt`
 - `echo "INFO: Processo concluído." >> log_sistema.txt`
 - `echo "AVISO: Disco cheio." >> log_sistema.txt`
- Use `grep` para encontrar todas as linhas que contenham "ERRO" em `log_sistema.txt`
- Use `find` para encontrar todos os arquivos com extensão `.txt` dentro de `exercicios_linux` e seus subdiretórios.

6. Limpeza

- Remova o arquivo `config.old`
- Remova o diretório `documentos` e todo o seu conteúdo.
- Volte para sua home e remova o diretório `exercicios_linux` e **todo o seu conteúdo** (use o parâmetro que remove diretórios não vazios).

6. Exercícios Práticos II

Cenário: organização de uma biblioteca digital de e-books e resenhas.

1. Preparação e primeira exploração

- Crie a estrutura do projeto de uma vez, sem usar `mkdir` várias vezes: `mkdir -p biblioteca/livros biblioteca/resenhas`
- Navegue até `biblioteca` e confirme sua localização com `pwd`.
- Liste o conteúdo com `ls -l` — repare que as pastas aparecem vazias.

2. Criando os arquivos (com verificação visual)

- Sem sair de `biblioteca`, crie três arquivos vazios dentro de `livros`, usando o caminho completo: `touch livros/dom_casmurro.txt livros/1984.txt livros/o_hobbit.txt`
- Confirme com `pwd` que você continua em `biblioteca` (não foi necessário entrar em `livros`).
- Liste tudo recursivamente com `ls -R` para ver a árvore completa de uma vez.

3. Preenchendo conteúdo e copiando com feedback

- Adicione uma resenha ao `dom_casmurro.txt` (dentro de `livros`) usando `echo "Nota: 9/10" > livros/dom_casmurro.txt` e depois `echo "Autor: Machado de Assis" >> livros/dom_casmurro.txt`
- Copie esse arquivo para dentro de `resenhas`, usando a flag `-v` para ver a confirmação: `cp -v livros/dom_casmurro.txt resenhas/`
- Exiba o conteúdo com `cat resenhas/dom_casmurro.txt` para conferir se copiou certo.

4. Renomeando e movendo com confirmação

- Renomeie `livros/1984.txt` para `livros/1984_george_orwell.txt` usando `mv`, com a flag `-v` para ver o que aconteceu: `mv -v livros/1984.txt livros/1984_george_orwell.txt`
- Mova `livros/o_hobbit.txt` para dentro de `resenhas` usando `mv -i livros/o_hobbit.txt resenhas/` — antes de mover, pratique a confirmação interativa (aqui não vai haver conflito, mas é bom criar o hábito).
- Liste `resenhas` com `ls -lh resenhas/` e observe o tamanho dos arquivos.

5. Editando direto no terminal

- Use `nano livros/1984_george_orwell.txt` para abrir o arquivo e adicionar duas linhas: título e uma frase marcante do livro. Salve e saia.
- Use `head -n 1 livros/1984_george_orwell.txt` para conferir só a primeira linha.
- Use `tail -n 1 livros/1984_george_orwell.txt` para conferir só a última linha.

6. Simulando um log de empréstimos e filtrando

- Ainda dentro de `biblioteca`, crie `emprestimos.log` e adicione, uma linha por vez:
`echo "EMPRESTADO: Dom Casmurro - Ana" > emprestimos.log`
`echo "DEVOLVIDO: 1984 - Carlos" >> emprestimos.log`
`echo "ATRASADO: O Hobbit - Beatriz" >> emprestimos.log`
`echo "EMPRESTADO: 1984 - Diego" >> emprestimos.log`
- Use `grep -i` (ignorando maiúsculas/minúsculas) para buscar todas as linhas com "atrasado":
`grep -i "atrasado" emprestimos.log`
- Use `grep -n` para mostrar o número da linha de cada ocorrência de "EMPRESTADO": `grep -n "EMPRESTADO" emprestimos.log`
- Use `find` para localizar todos os arquivos `.txt` dentro de `biblioteca`, combinando com `wc -l` via pipe para contar quantos são: `find . -name "*.txt" | wc -l`

7. Limpeza com confirmação (diferente da Versão 1 — usa -i em vez de remoção direta)

- Remova `emprestimos.log` usando `rm -i emprestimos.log` e confirme a exclusão quando perguntado.
- Remova o diretório `resenhas` inteiro com `rm -r resenhas`.
- Volte para a home e remova `biblioteca` por completo, com `rm -r biblioteca` (o parâmetro `-r` é obrigatório, pois a pasta ainda contém `livros` com arquivos dentro).